Python Exercises

Basic concepts hands-on questions

Installing Python & Running Programs

1. **Simple Hello World**
   - **Objective:** Create first Python file.
   - **Task:** Write and execute hello program.
   - **Instructions:**
     - Create first.py with print("Hello World!").
     - Run ***python first.py***.
     - Share terminal output.

Text Editors and IDEs

2. **Jupyter Notebook**
   - **Objective:** Create Jupyter notebook.
   - **Task:** Basic notebook operations.
   - **Instructions:**
     - Run ***pip install notebook***, then ***jupyter notebook***.
     - Create new notebook, add code/math cells.

3. **VS Code Setup**
   - **Objective:** Configure Python in VS Code.
   - **Task:** Install Python extension.
   - **Instructions:**
     - Install VS Code Python extension.
     - Create .py file, verify IntelliSense.
     - Run with Ctrl+F5.

Data Types

4. **Float Precision**
   - **Objective:** Handle floating-point arithmetic with functions + validation + formatted output.
   - **Task:** Calculate net salary after tax and print with 2 decimals.
   - **Instructions:**
     - salary = 75000.5, tax_rate = 0.18.
     - Use a function
     - Validate salary & tax rate
     - Use variables
     - Format output to 2 decimal places

Variables

5. **Multiple Assignment**
   - **Objective:** Use multiple assignment with basic validation + function + formatted output.
   - **Task:** Unpack (x, y) coordinates and display them.
   - **Instructions:**
     - Use a function
     - Validate coordinates
     - Use multiple assignment
     - Print coordinates

Mathematical Operations

6. **Modulo Operator**
   - **Objective:** Use the modulo operator with function + validation + clean output.
   - **Task:** Check if a number is even or odd using %.
   - **Instructions:**
     - Use a function
     - Validate number
     - Compute remainder
     - Print "Even" or "Odd"
     - Use variable number = 17

7. **Floor Division**
   - **Objective:** Perform integer division with function + validation + formatted output.
   - **Task:** Split a total bill equally among people.
   - **Instructions:**
     - Use a function
     - Validate bill & number of people
     - Use floor division //
     - Print individual share
     - Variables: total_bill = 1250, people = 4

8. **Min/Max Functions**
   - **Objective:** Use built-in min() and max() with validation + function + structured output.
   - **Task:** Find the highest and lowest salary in a list.
   - **Instructions:**
     - Use a function
     - Validate the salary list
     - Compute min & max
     - Print the results
     - Use list: [50000, 75000, 62000, 95000]

User Input

9. **Basic Input**
   - **Objective:** Read and validate user input using a function + formatted output.
   - **Task:** Get the user's name and greet them.
   - **Instructions:**
     - Use a function

    - Validate empty input
    - Use input()
    - Print greeting message

10. **Numeric Input**
    - **Objective:** Convert user input to a number with validation + function + formatted output.
    - **Task:** Ask user for age, validate, convert to integer, and print next year's age.
    - **Instructions:**
        - Use a function
        - Validate numeric input
        - Convert using int()
        - Print "Next year you'll be <age+1>"

11. **Float Input**
    - **Objective:** Handle decimal input with validation + function + formatted output.
    - **Task:** Convert weight from kilograms to pounds.
    - **Instructions:**
        - Use a function
        - Validate decimal input
        - Convert kg → lbs using "lbs = kg * 2.20462"
        - Print the weight in pounds.

Control Flow

12. **Simple If**
    - **Objective:** Basic conditional with function, validation, and user interaction.
    - **Task:** Check if a student passes based on marks.
    - **Instructions:**
        - Use a function
        - Validate marks
        - Use a variable (marks = 75)
        - Print Pass/Fail

13. **If-Else**
    - **Objective:** Two-way decision using functions + validation + formatted output.
    - **Task:** Check if a number is even or odd.
    - **Instructions:**
        - Use a function
        - Validate input
        - Use variable num = 8
        - Print "Even" or "Odd"

14. **If-Elif-Else**
    - **Objective:** Multi-way decision using functions + validation + structured output.
    - **Task:** Assign a grade based on the score.
    - **Instructions:**
        - Use a function
        - Validate score
        - Use variable score = 88

    - Print grade A/B/C
    - Add slight enhancements over the basic version

15. **Nested If**
    - **Objective:** Use nested conditions with a function + basic validation + formatted output.
    - **Task:** Validate username and password using nested if statements.
    - **Instructions:**
        - Use a function
        - Validate blank inputs
        - Nested if for user and password
        - Variables:
            - user = "admin"
            - pwd = "pass123"

16. **For Loop Basics**
    - **Objective:** Iterate a fixed number of times with a function + validation + formatted output.
    - **Task:** Print numbers from 1 to 5 using range().
    - **Instructions:**
        - Use a function
        - Validate loop count
        - Use for i in range(5)
        - Print numbers 1–5

17. **While Loop**
    - **Objective:** Use a condition-based loop with validation + function + formatted output.
    - **Task:** Countdown from 5 to 1 using a while loop.
    - **Instructions:**
        - Use a function
        - Validate count value
        - Use while count > 0:
        - Print count each step
        - Decrease using count -= 1

18. **Break Statement**
    - **Objective:** Use a break statement to exit a loop early with function + validation + clean output.
    - **Task:** Find and print the first even number in a given range.
    - **Instructions:**
        - Use a function
        - Validate the range size
        - Loop with for i in range(...)
        - Use modulo % to check even
        - Break once the first even is found

19. **Continue Statement**
    - **Objective:** Use continue to skip even numbers, with a function + validation + formatted output.
    - **Task:** Sum only the odd numbers in a given range.
    - **Instructions:**

    - Use a function
    - Validate the range
    - Loop with range(10)
    - Skip even numbers using continue
    - Accumulate only odd numbers

20. **Pass Statement**
    - **Objective:** Use pass as a placeholder inside a function, with simple structure and formatted output.
    - **Task:** Define an empty function using pass and print confirmation.
    - **Instructions:**
        - Use a function
        - Add placeholder logic with pass
        - Call the function
        - Print "Function defined"

Whitespace Importance

21. **Consistent Indentation**
    - **Objective:** Demonstrate clean, consistent indentation using a nested if inside a function.
    - **Task:** Print "Nested" when both conditions are True.
    - **Instructions:**
        - Use a function
        - Apply exactly 4 spaces per indentation level
        - Use nested if-statements
        - Print "Nested"
        - Print confirmation message

Organizing Programs

22. **Comment Usage**
    - **Objective:** Show proper documentation using meaningful comments + a function.
    - **Task:** Add explanatory comments and print the total salary.
    - **Instructions:**
        - Use clear comments
        - Use a function
        - Define base and bonus
        - Calculate total
        - Print result

Modules

23. **Import Standard Module**
    - **Objective:** Use the math module inside a function with validation + formatted output.
    - **Task:** Calculate the area of a circle using math.pi and a given radius.
    - **Instructions:**
        - Import math
        - Use a function
        - Validate radius

    - Use formula: area = math.pi × radius^2
    - Print area with 2 decimals

24. **All Import**
    - **Objective:** Use from module import * along with simple utility usage inside a function.
    - **Task:** Import everything from a standard module and use a few functions.
    - **Instructions:**
        - Use from math import *
        - Use a function
        - Validate some input
        - Demonstrate usage (e.g., sqrt, pow, pi)
        - Print results

Functions

25. **Parameters**
    - **Objective:** Pass arguments into a function with validation + structured output.
    - **Task:** Create a function that adds two numbers and prints the result.
    - **Instructions:**
        - Use a function with parameters
        - Validate inputs
        - Return the sum
        - Print the result of add(5, 3)

26. **Multiple Parameters**
    - **Objective:** Handle multiple arguments in a function with validation + formatted output.
    - **Task:** Calculate the area of a rectangle using length and width.
    - **Instructions:**
        - Use a function that accepts two arguments
        - Validate the numbers
        - Calculate and return area
        - Print area(5, 3)

Built-in Functions

27. **Len Function**
    - **Objective:** Use len() inside a function with basic validation + structured output.
    - **Task:** Get the length of a given string.
    - **Instructions:**
        - Use a function
        - Validate the string
        - Use len(text)
        - Print the length

File I/O

28. **Write to File**
    - **Objective:** Create and write to a text file using a function + clean structure.
    - **Task:** Save a greeting message into a text file.
    - **Instructions:**
        - Use a function
        - Open a file in write mode
        - Write "Hello World" into the file
        - Print confirmation message

29. **Read from File**
    - **Objective:** Read text from a file using a function with basic error handling.
    - **Task:** Open a text file and display its contents.
    - **Instructions:**
        - Use a function
        - Open file in read mode
        - Read full content using read()
        - Print file content
        - Handle missing file gracefully

Error Handling

30. **Basic Try-Except**
    - **Objective:** Catch simple runtime errors using try-except with a function and clear output.
    - **Task:** Safely divide two numbers and handle division-by-zero errors.
    - **Instructions:**
        - Use a function
        - Use try-except
        - Handle ZeroDivisionError
        - Print result or error message

Lists

31. **Create List**
    - **Objective:** Initialize a list with basic validation + a function + clean output.
    - **Task:** Create a shopping cart list and display the items.
    - **Instructions:**
        - Use a function
        - Validate list items
        - Initialize list with values [100, 250, 75]
        - Print the cart contents

32. **Append to List**
    - **Objective:** Add a single item to a list using a function with validation.
    - **Task:** Add a new expense to an existing expenses list.
    - **Instructions:**
        - Use a function
        - Validate the expense amount

    - Use .append()
    - Print the updated expenses list

Dictionaries

33. **Update Dictionary**
    - **Objective:** Perform bulk dictionary updates using a function with basic validation.
    - **Task:** Merge employee details from two dictionaries.
    - **Instructions:**
        - Use a function
        - Validate dictionary inputs
        - Use .update()
        - Print the updated employee data

34. **Nested Dictionary**
    - **Objective:** Work with nested dictionaries using a function and basic validation.
    - **Task:** Store department-wise employee data and retrieve a specific employee's salary.
    - **Instructions:**
        - Use a nested dictionary
        - Use a function to fetch data
        - Validate department and employee name
        - Print the employee salary

Tuples and Sets

35. **Create Tuple**
    - **Objective:** Use an immutable tuple with a function and basic validation.
    - **Task:** Store fixed coordinates and display them.
    - **Instructions:**
        - Use a tuple
        - Use a function
        - Validate coordinate values
        - Print coordinates in a readable format

36. **Set Intersection**
    - **Objective:** Find common elements between two sets using a function and basic validation.
    - **Task:** Identify common skills between two skill sets.
    - **Instructions:**
        - Use sets
        - Use a function
        - Validate inputs
        - Find intersection using &
        - Print common skills

OOP concepts

Classes

37. **Multiple Instances**
    - **Objective:** Create multiple objects from a class and access their attributes.
    - **Task:** Create an employee roster using multiple instances of a class.
    - **Instructions:**
        - Define a class with __init__
        - Create multiple objects (instances)
        - Add a method to display employee info
        - Print employee names

38. **Method Chaining**
    - **Objective:** Demonstrate method chaining by returning self from methods.
    - **Task:** Create an employee object, set salary, apply raise, and display final salary.
    - **Instructions:**
        - Create a class
        - Methods should return self
        - Use method chaining
        - Add basic validation
        - Print final salary

Custom Classes & Inheritance

39. **Polymorphism**
    - **Objective:** Demonstrate polymorphism using method overriding.
    - **Task:** Show different behaviors of the same method (work()) for different employee types.
    - **Instructions:**
        - Create a base class with a common method
        - Override the method in child classes
        - Store objects in a list
        - Call the same method for all objects
        - Observe different outputs

40. **Class Methods**
    - **Objective:** Use a class method as a factory to create objects from a formatted string.
    - **Task:** Create an Employee object from a string like "Shubh,75000".
    - **Instructions:**
        - Use @classmethod
        - Parse string input
        - Convert salary to integer
        - Return a new object using cls
        - Print employee details

Real-time simulation hands-on questions

41. **Employee Management System**
    - **Objective:** Use classes, dictionaries, lists, and JSON file I/O together.
    - **Task:** Create an employee management system that can save and load employee data.
    - **Instructions:**
        - Create Employee class with __init__ and __str__
        - Store employees in a dictionary
        - Save data to emps.json
        - Load data back from file
        - Print all employees

42. **Data Analysis Pipeline**
    - **Objective:** Use modules, functions, lists, file handling, and error handling together.
    - **Task:** Process sales data from a file and calculate statistics.
    - **Instructions:**
        - Import statistics module
        - Read numeric data from sales.txt
        - Handle file and data errors using try-except
        - Calculate mean and median
        - Print a statistics summary

43. **Configuration Manager**
    - **Objective:** Use classes, inheritance, dictionaries, and file I/O to manage application configuration.
    - **Task:** Create a configurable system that loads database settings from a file.
    - **Instructions:**
        - Create a base Config class
        - Create DatabaseConfig class inheriting from Config
        - Use configparser module
        - Load settings from db.ini
        - Validate required keys

44. **CSV Data Processor**
    - **Objective:** Use file I/O, lists, dictionaries, and comprehensions to analyze CSV data.
    - **Task:** Analyze employee salary data from a CSV file.
    - **Instructions:**
        - Read employees.csv using the csv module
        - Convert rows into a list of dictionaries
        - Filter employees with salary > 50000 using list comprehension
        - Calculate and print average salary

45. **Expense Tracker**
    - **Objective:** Use file handling, date processing, lists, dictionaries, and comprehensions for expense analysis.
    - **Task:** Analyze current month expenses and calculate totals per category.
    - **Instructions:**

- Read expenses.csv (date, amount, category)
- Filter current month using datetime
- Group expenses by category using dictionary
- Calculate total amount per category
- Print summary

46. **API Response Handler**
    - **Objective:** Use requests, JSON handling, classes, and error handling to process API responses.
    - **Task:** Fetch weather data from an API and display temperature and conditions safely.
    - **Instructions:**
        - Install & use requests
        - Fetch weather data (JSON)
        - Parse temperature and conditions
        - Handle 404 and network errors
        - Format output neatly

Additional real-time simulation hands-on questions

47. **Complete Calculator Program**
    - **Objective:** Combine input, functions, error handling.
    - **Task:** Build a robust calculator that supports + , - , * , /.
    - **Instructions:**
        - Create function calculate(a, b, op) handling +, -, *, /.
        - Use input() for numbers/operation.
        - Handle ZeroDivisionError and ValueError.
        - Print formatted result.

48. **Shopping Cart System**
    - **Objective:** Use classes, lists, control flow, and the math module to build a simple shopping cart system.
    - **Task:** Perform complete cart operations and print a receipt with GST.
    - **Instructions:**
        - Create CartItem class with price and quantity
        - Create ShoppingCart class with:
            - add item
            - remove item
            - calculate total
        - Apply 18% GST
        - Print final receipt

49. **Temperature Converter GUI**
    - **Objective:** Use classes, user input, modules, and formatted output.
    - **Task:** Convert temperatures between Celsius, Fahrenheit, and Kelvin.
    - **Instructions:**
        - Converter class

- Convert C ↔ F ↔ K
- Use input() menu
- Format output to 2 decimals
- Use tabulate module for table display

50. **Backup Utility**
    - **Objective:** Use file operations, modules, sets, and error handling to build a smart backup tool.
    - **Task:** Copy files to a backup folder while skipping duplicates and logging operations.
    - **Instructions:**
        - Use shutil module.
        - Copy files but skip duplicates using sets.
        - Handle FileNotFoundError, PermissionError.
        - Log operations to backup.log.

51. **URL Shortener**
    - **Objective:** Use dictionaries, hashing, classes, and modules to build a simple URL shortening service.
    - **Task:** Create a basic URL shortener that can shorten URLs and retrieve the original URL.
    - **Instructions:**
        - Import hashlib
        - URLShortener class with dictionary storage
        - Generate 6-character hash from URL
        - Lookup (redirect simulation) functionality

52. **Gradebook System**
    - **Objective:** Use dictionaries, functions, control flow, and file I/O for student grade management.
    - **Task:** Manage student grades, calculate GPA, save/load data, and compute class average.
    - **Instructions:**
        - Dictionary: student → list of grades
        - Functions: add grade, calculate GPA, save/load
        - Handle invalid grade inputs
        - Print class average

53. **Task Scheduler**
    - **Objective:** Use classes, the datetime module, lists, and sorting to manage tasks.
    - **Task:** Create a task scheduler that prioritizes tasks by due date and identifies overdue tasks.
    - **Instructions:**
        - Task class with due_date and priority
        - Store tasks in a list
        - Sort tasks by due date using datetime
        - Filter overdue tasks
        - Print sorted task schedule

54. **Inventory Manager**
    - **Objective:** Use classes, inheritance, dictionaries, and sets to manage warehouse inventory.

- **Task:** Track products, stock levels, and generate low-stock alerts.
- **Instructions:**
  - Product base class
  - Perishable and Electronics inherit from Product
  - Track stock levels using a dictionary
  - Use sets for low-stock alerts
  - Print inventory summary

55. **Budget Planner**

- **Objective:** Use classes, control flow, external modules, and visualization to track a monthly budget.
- **Task:** Create a monthly budget planner that tracks spending, alerts overspending, and visualizes data.
- **Instructions:**
  - Category class with limit and spent
  - Input expenses using a loop
  - Use matplotlib for pie chart visualization
  - Alert when category exceeds budget